

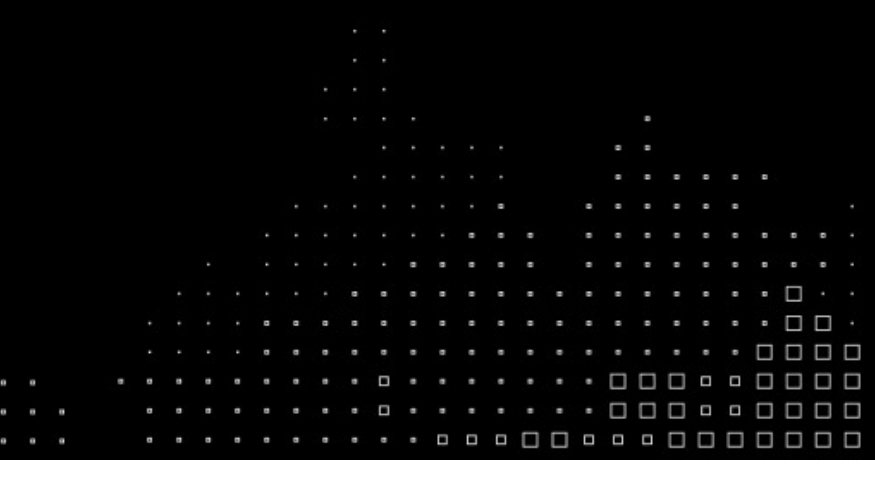
FIAP

NBA

MBA⁺

**Engenharia
de Software**

Aula 2



Thiago Keller – 40 anos

Software Engineer @ **MercadoLivre** | | Professor / Scrum Master @ **FIAP**

- Tecnologia da Informação pela **UBC – Universidade Braz Cubas**.
- MBA Gestão de TI - **Anhanguera Educacional**.
- MBA Engenharia de Software - **Impacta**.
- MBA Arquitetura de Soluções - **FIAP**.
- MBA NeoLiderança e Disrupção Organizacional – **Coimbra Business School – Portugal – 2024**
- **Mestrando em Computação Aplicada Mackenzie**

Início em tecnologia em 2006. São **20 anos** de experiência em TI nas empresas: Acai, TQS, Itaú, Stone



/thiago-keller



profthiago.vicco@fiap.com.br

Trajetória

- **Docente e Scrum Master** - Análise e Desenvolvimento de Sistemas
 - Responsável por StartupOne do 2TDS
 - Vice coordenador do curso
- **Docente MBA** – Engenharia de Software

Curiosidades sobre mim:

- Colecionador de Vídeo Games (20 consoles)
- 20 tatuagens
- Nerd assumido
- Pai da Eduarda – Dezembro 2024



/thiago-keller



profthiago.vicco@fiap.com.br

MBA Coimbra - Portugal



<https://www.fiap.com.br/mba/extensao-internacional/coimbra/neolideranca-e-disrupcao-organizacional/>

Combinados

Aulas

- Inicio - 19h30
- Intervalo – 21h00
- Retorno – 21h20
- Termina - 23h00

Experience Survey

Por favor após a aula respondam a nossa
pesquisa de satisfação

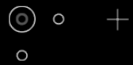


Qual é o seu propósito ?

**FAZER O MEU MELHOR, NA CONDIÇÃO QUE EU TENHO. ENQUANTO NÃO TENHO CONDIÇÕES MELHORES PRA
FAZER MELHOR AINDA!!**

<https://www.linkedin.com/in/fabiano-gomes-8068a820/>

Revisão Rápida da Aula 1



Definição:

Alinhar o desenvolvimento de software com as necessidades do **negócio**.

Objetivo:

Atacar a complexidade do **domínio**



Complexidade de Software



+ Quais são os benefícios do DDD ? +

Comunicação
Efetiva

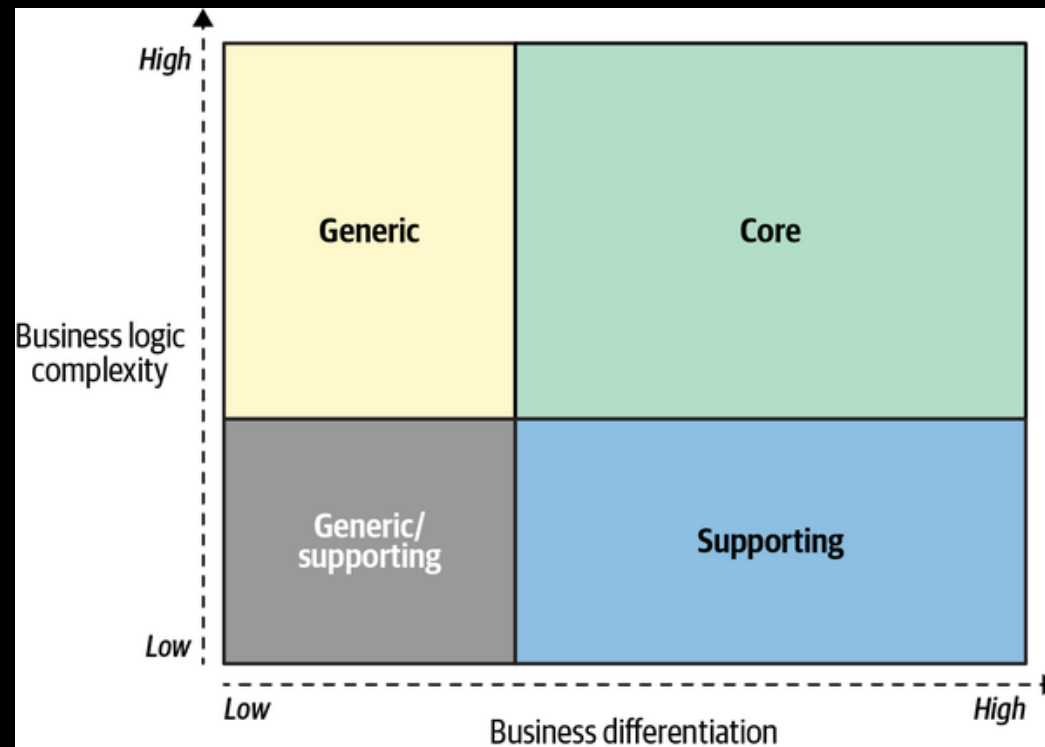
Flexibilidade
e
Manutenção

Maior
Resiliência

Modelagem
Rica

Soluções que
Agregam
Valor

Gráfico



<https://www.branas.io/blog/relacao-entre-subdomain-e-bounded-context-no-domain-driven-design.html>

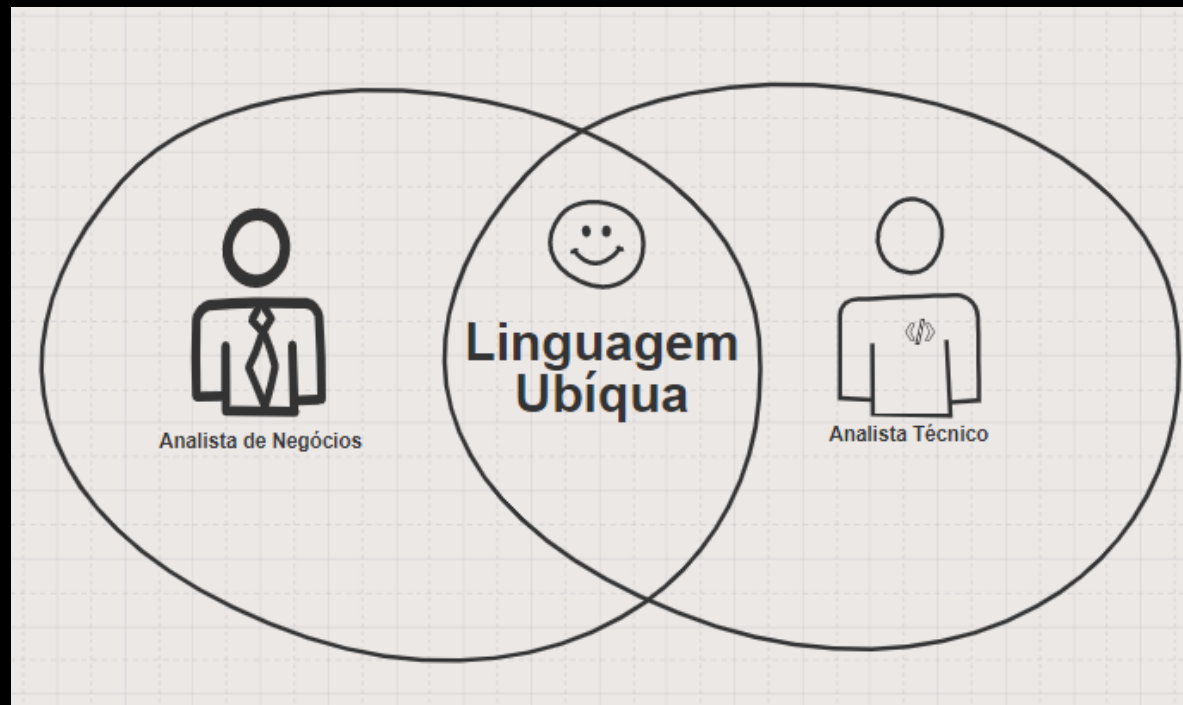
Design Estratégico

O design estratégico trata da visão **macro** do domínio e como dividimos o sistema em **bounded contexts** (contextos delimitados). Ele responde a perguntas como:

- a) Quais são os bounded contexts?
- b) Quais subdomínios são estratégicos, de suporte e genéricos?
- c) Como os bounded contexts se comunicam?



Linguagem ubíqua/onipresente



Keller's Health

Termo	Descrição
Consulta	Sessão médica entre paciente e médico, podendo incluir chamada de vídeo, receita ou atestado.
Paciente	Usuário que agenda e realiza consultas na plataforma.
Médico	Profissional de saúde que realiza as consultas.
Prontuário	Registro com o histórico médico do paciente, incluindo diagnósticos e receitas.
Receita	Prescrição médica emitida durante uma consulta.
Pagamento	Transação financeira realizada pelo paciente para completar uma consulta.
Disponibilidade	Horários livres dos médicos para agendamento de consultas.

“Modelos de Domínio verdadeiramente bons evoluem com o tempo.”

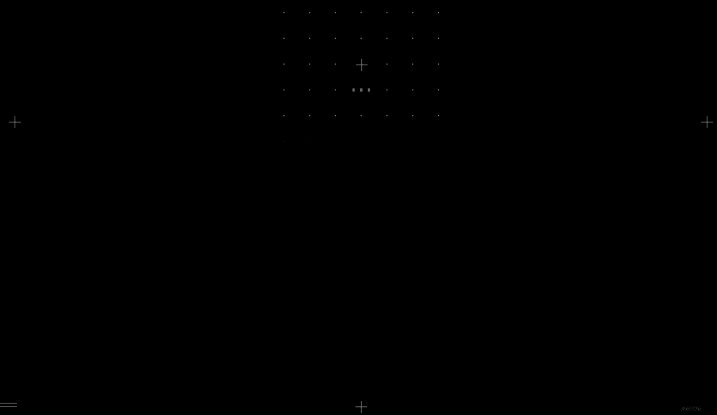
Eric Evans

ALGUMA PERGUNTA?

(POR FAVOR NÃO)

WWW.GERARMEMES.COM.BR

Dinâmica 1



Voluntário para apresentação?

+ + + + +
 + + + + +
 + + + + +
 + + + + +
 + + + + +
 + + + + +

+

+

—

+

—

Aula 2

+

+ + + + +
 + + + + +
 + + + + +
 + + + + +
 + + + + +
 + + + + +

+

—

+

—

Bounded Context

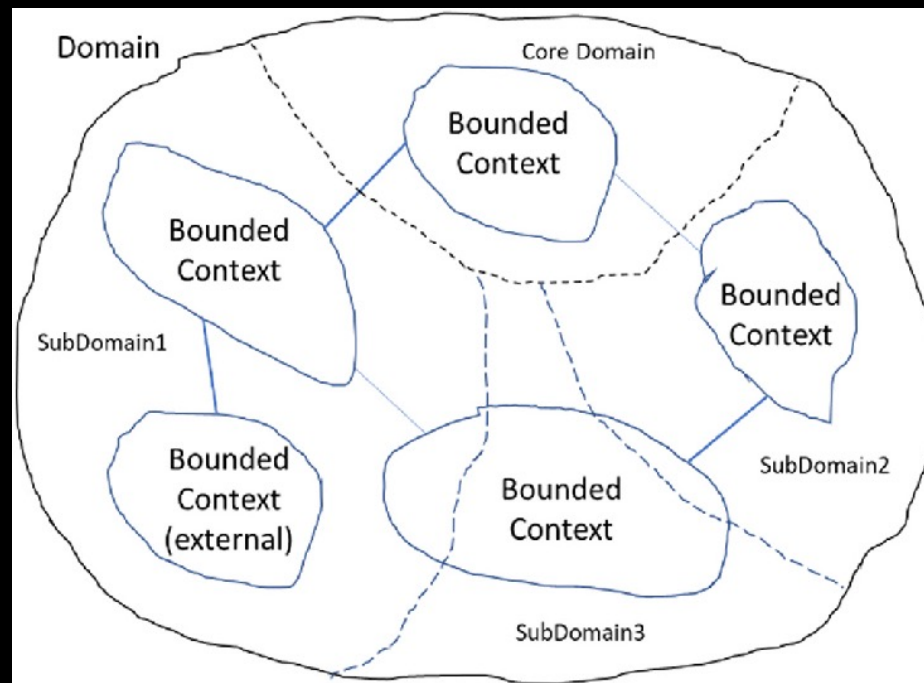
(Contexto Delimitado)

- Cada **Bounded Context** tem sua própria linguagem ubíqua.
- Ele **delimita** onde um conjunto de conceitos faz sentido.
- Ele **protege** um modelo de domínio específico, evitando ambiguidades.
- Pode se comunicar com outros contextos, mas mantém sua independência.

<https://github.com/ddc-crew/context-mapping>



Bounded Context



<https://github.com/ddd-crew/context-mapping>



Context Mapping

(Mapeamento de Contexto)

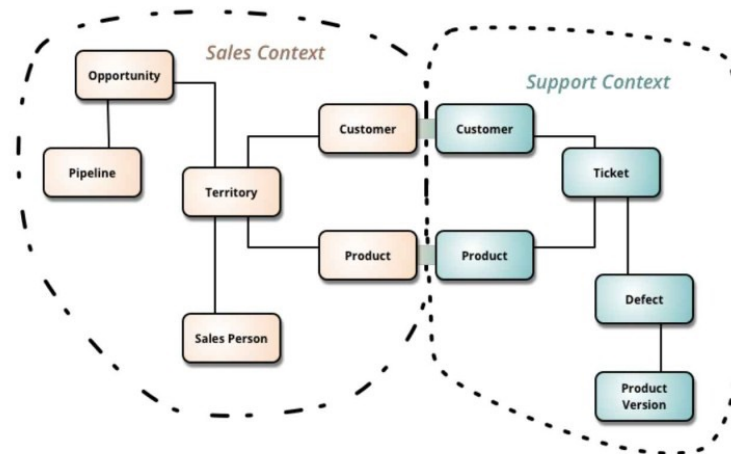
Como os Bounded Contexts interagem entre si

- Ajuda a **entender dependências** entre contextos.
- Define **estratégias de comunicação** entre times e sistemas.
- Ajuda a **evitar alto acoplamento** e **melhorar a autonomia** das equipes.



Context Mapping

Bounded Contexts



DDD deals with large models by dividing them into different Bounded Contexts and being explicit about their interrelationships.

<http://martinfowler.com/bliki/BoundedContext.html>



Tipos de Relacionamento

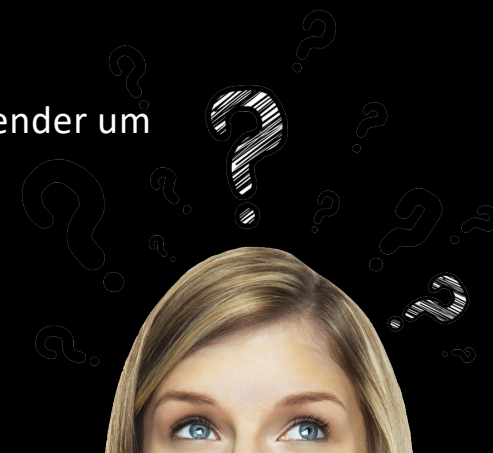
No Context Mapping, usamos **padrões** para definir como os **Bounded Contexts** interagem

Parceria (Partnership)

- Dois contextos trabalham juntos e **dependem um do outro**.
- Mudanças em um **afetam diretamente o outro**.

Exemplo: Contexto de Agendamento e um Contexto de Consultas podem depender um do outro para sincronizar horários.

Risco: Se um time falhar, o outro pode ser prejudicado.



Tipos de Relacionamento

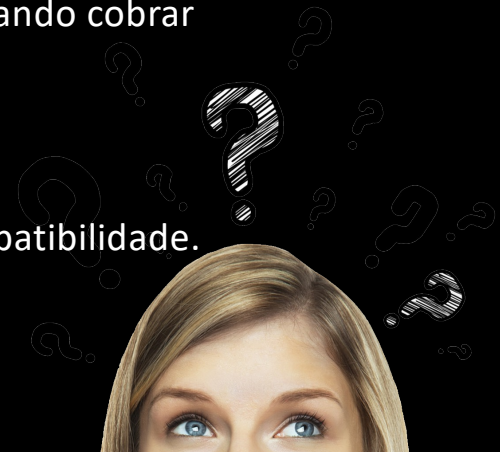
Cliente-Fornecedor (Customer-Supplier)

Um contexto cliente depende dos dados ou serviços de um contexto fornecedor.

O fornecedor pode operar independentemente, mas mudanças nele afetam o cliente.

Exemplo: O Contexto de Pagamentos depende do Contexto de Consultas para saber quando cobrar uma consulta finalizada.

Solução: O cliente precisa acompanhar mudanças no fornecedor para garantir compatibilidade.



Tipos de Relacionamento

Kernel Compartilhado (Shared Kernel)

- Dois contextos compartilham um pequeno conjunto de regras ou dados essenciais.



Exemplo: O Contexto de Cadastro de Usuários pode compartilhar um modelo básico de "Usuário" com o Contexto de Autenticação.

Risco: Se o Shared Kernel crescer demais, vira um monólito disfarçado.



Tipos de Relacionamento

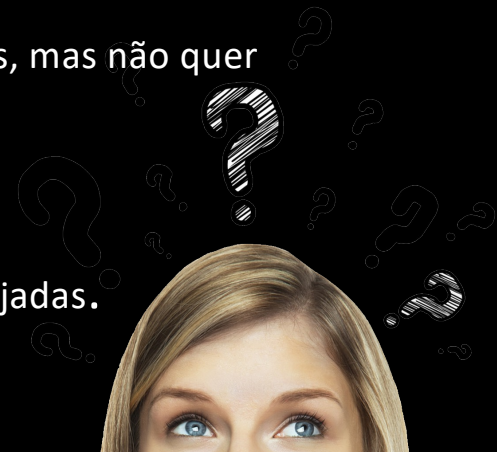
Tradutor/Anticorruption Layer (ACL - Anti-Corruption Layer)

- Um contexto usa um adaptador para traduzir dados recebidos de outro contexto e evitar impacto direto.



Exemplo: O Contexto de Consultas⁺ pode precisar de dados do Contexto de Pagamentos, mas não quer depender diretamente do modelo de "Fatura".

Vantagem: Protege o modelo de domínio contra mudanças externas indesejadas.



Tipos de Relacionamento

Conformista (Conformist)

- Um contexto aceita o modelo do outro sem questionar.

Exemplo: Se o Contexto de Relatórios precisa de dados do Contexto de Consultas, ele pode apenas consumir as APIs disponíveis, sem tentar modificar nada.

Risco: Perda de controle sobre as regras de negócio, pois o conformista depende totalmente do outro contexto.



Upstream x Downstream

O que são?

Upstream → é quem **fornece** a informação (produtor).

Downstream → é quem **consome** a informação (dependente).



Exemplo

Contextos:

- **Agendamento Context** (responsável por consultas médicas).
- **Notificação Context** (responsável por avisar pacientes via SMS/Email).

Fluxo:

- No Agendamento, o paciente marca uma consulta → gera evento `ConsultaAgendada`.
- O evento é publicado.
- O contexto de Notificação consome esse evento para disparar mensagens.
- **Agendamento = Upstream** (origem da informação).
- **Notificação = Downstream** (dependente do evento).



Tipos de relacionamento

Conformist

- O Downstream aceita exatamente o modelo do Upstream.
- Pouca liberdade para customizar.
- Exemplo: Notificação usa o mesmo DTO de ConsultaAgendada do Agendamento.



Anti-Corruption Layer (ACL)

- O Downstream cria uma camada de tradução.
- Protege o modelo interno de ser “contaminado”.
- Exemplo: Notificação recebe ConsultaAgendadaDTO, mas converte para sua própria entidade MensagemNotificacao.



The Kahoot! logo is centered on a world map background. The map is divided into four quadrants by a vertical and a horizontal line. The top-left quadrant is red, the top-right is blue, the bottom-left is yellow, and the bottom-right is green. The word "Kahoot!" is written in a large, white, bold, sans-serif font across the center of the map.

Kahoot!

Dinâmica 2:

DDD - 2



Design Tático

Ajuda a modelar **entidades, regras de negócio e interações** dentro de um **Bounded Context**.

entidades
regras de negócio
interações



Design Tático

- Entidades
- Value Objects
- Agregados
- Domínio Rico vs. Anêmico
 - Repositórios
 - Serviços de Domínio
 - Eventos de Domínio



Design Tático

Entidades

São objetos que possuem **identidade única** no sistema.

Mesmo que duas entidades tenham os **mesmos atributos**, se os IDs forem diferentes, são **objetos distintos**.



Design Tático

Value Objects

São objetos que não possuem identidade própria.

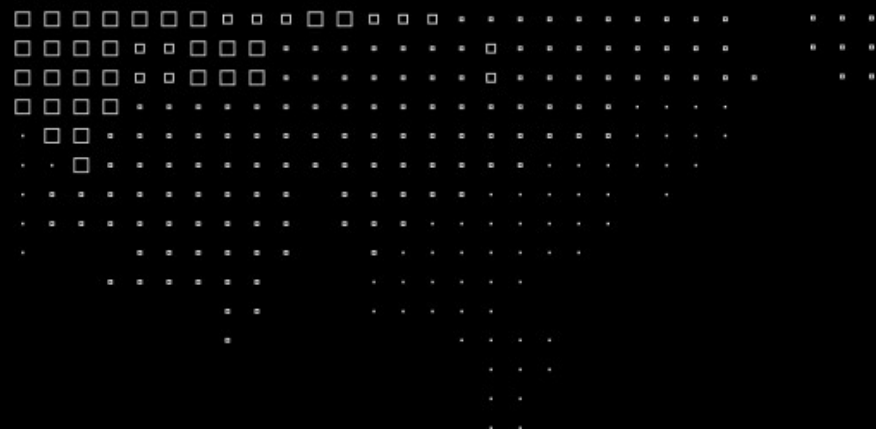
Dois objetos com os **mesmos valores** são considerados iguais.

São **imutáveis** e usados para representar conceitos do domínio.



FIAP

NBA


MBA⁺

Copyright © **2024** Prof. Thiago Keller

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).

